

sca_ML

September 22, 2026

This project aims to benchmark a simple linear regression model to a lasso regression model with hyperparameter search space, By recreating the linear regression model that is introduced in MITx Supply Chain Management Micromasters / SCO Supply Chain Analytics Course using the same underlying supply chain (transportation) data, BUT in two ways:

1. ML-1 (Linear Regression): Applying the same conditions (no scaling, etc.) to the model as applied in the course, except the possibilities of
 - second degree polynomial structure use with interactions allowed between features,
 - converting lead time into a ordinal / binary feature,
 - restriction of coefficients to being positive only.
2. ML-2 (Lasso Regression): Applying full-fledged hyperparametrization including polynomial degree, feature interaction, scaling and coefficient regularization.

KEY FINDING: ML-1, despite being simpler than the other, was slightly better than ML-2 on both train and test R^2 scores and residuals.

- ML-1 Train R^2 score (CV): 0.84
- ML-1 Validation R^2 score: 0.80
- ML-1 Train R^2 score: 0.83
- **ML-1 Test R^2 score: 0.72**
- **ML-1 Test R^2 score (adjusted): 0.71**

Note: “cpl” is the short for “Cost per Load” and it is the target feature in this project

1 Import

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display

plt.rc("axes.spines", top=False, right=False)
plt.style.use("ggplot")

from sklearn.model_selection import train_test_split, GridSearchCV,
    cross_validate
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression, LassoCV
```

```
from sklearn.pipeline import Pipeline
from sklearn.metrics import r2_score
```

2 Load data

```
[2]: df = pd.read_csv("duffy.csv")
df = df.drop(columns = "Number ID")
```

```
[3]: df.head()
```

```
[3]:
```

	CPL	Dist	LdTime	TrlLng	Wgt	Equpt
0	3692	1579	1	53	20559	DRY
1	3279	1298	12	48	17025	REF
2	3120	1382	11	48	26735	DRY
3	3205	1033	1	53	26175	DRY
4	3188	1320	3	53	17994	DRY

```
[4]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   CPL          100 non-null    int64
1   Dist         100 non-null    int64
2   LdTime       100 non-null    int64
3   TrlLng       100 non-null    int64
4   Wgt          100 non-null    int64
5   Equpt        100 non-null    str
dtypes: int64(5), str(1)
memory usage: 5.1 KB
```

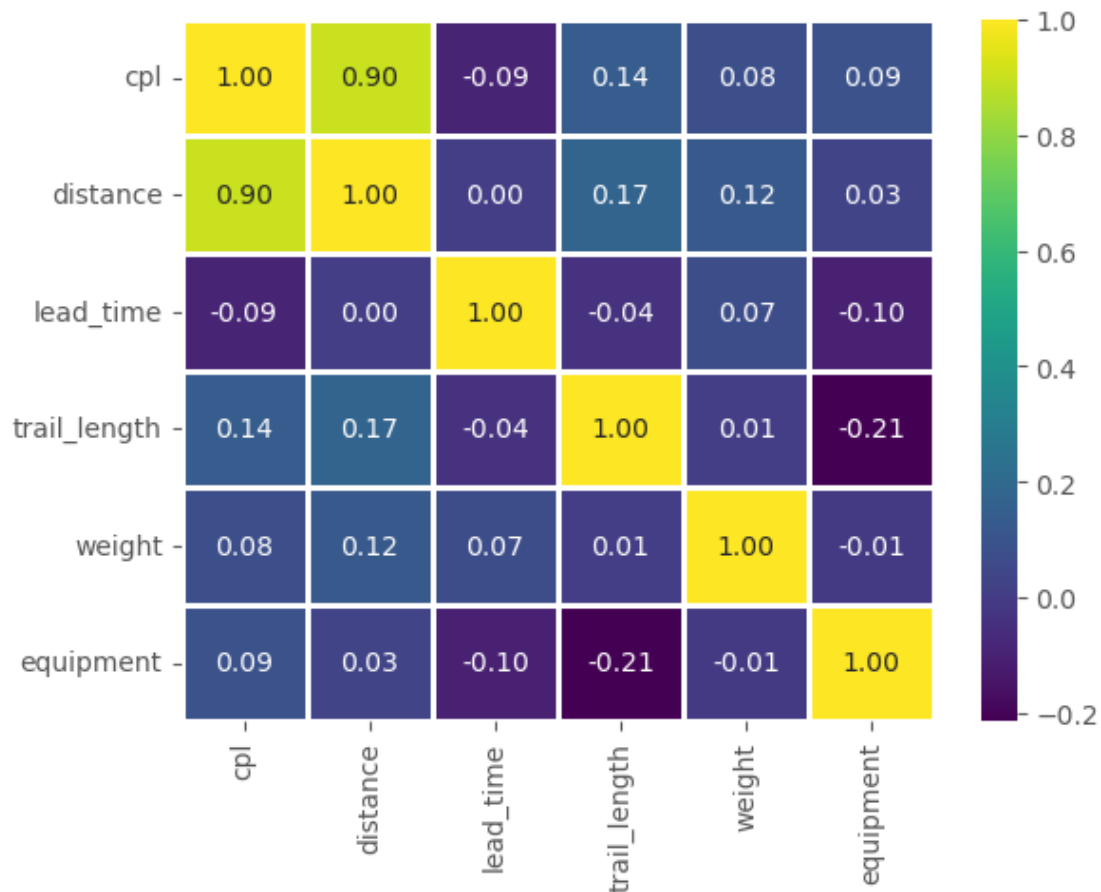
2.1 Manipulate Data

```
[5]: df.columns = ["cpl", "distance", "lead_time", "trail_length", "weight", "equipment"]
df["trail_length"] = df.trail_length.apply(lambda cell: 0 if cell == 48 else 1)
df["equipment"] = df.equipment.apply(lambda cell: 0 if cell == "DRY" else 1)
```

- Trail length and equipment values are rendered binary in a way to contribute to the cpl positively if 1.

2.2 Multicollinearity check

```
[6]: sns.heatmap(  
    data=df.corr(),  
    annot=True,  
    fmt=".2f",  
    ec="white",  
    lw=1,  
    cmap="viridis",  
    snap=True  
);
```



- No multicollinearity issue between the features.

2.3 Specific data manipulation

```
[7]: df["lead_time1"] = -df.lead_time
```

- Lead_time will never be used in the models below at all. Instead, lead_time1 will be used.

- Lead_time1: actual values are rendered negative to force ML models below to produce positive coefficients only.

```
[8]: df.head()
```

```
[8]:
```

	cpl	distance	lead_time	trail_length	weight	equipment	lead_time1
0	3692	1579	1	1	20559	0	-1
1	3279	1298	12	0	17025	1	-12
2	3120	1382	11	0	26735	0	-11
3	3205	1033	1	1	26175	0	-1
4	3188	1320	3	1	17994	0	-3

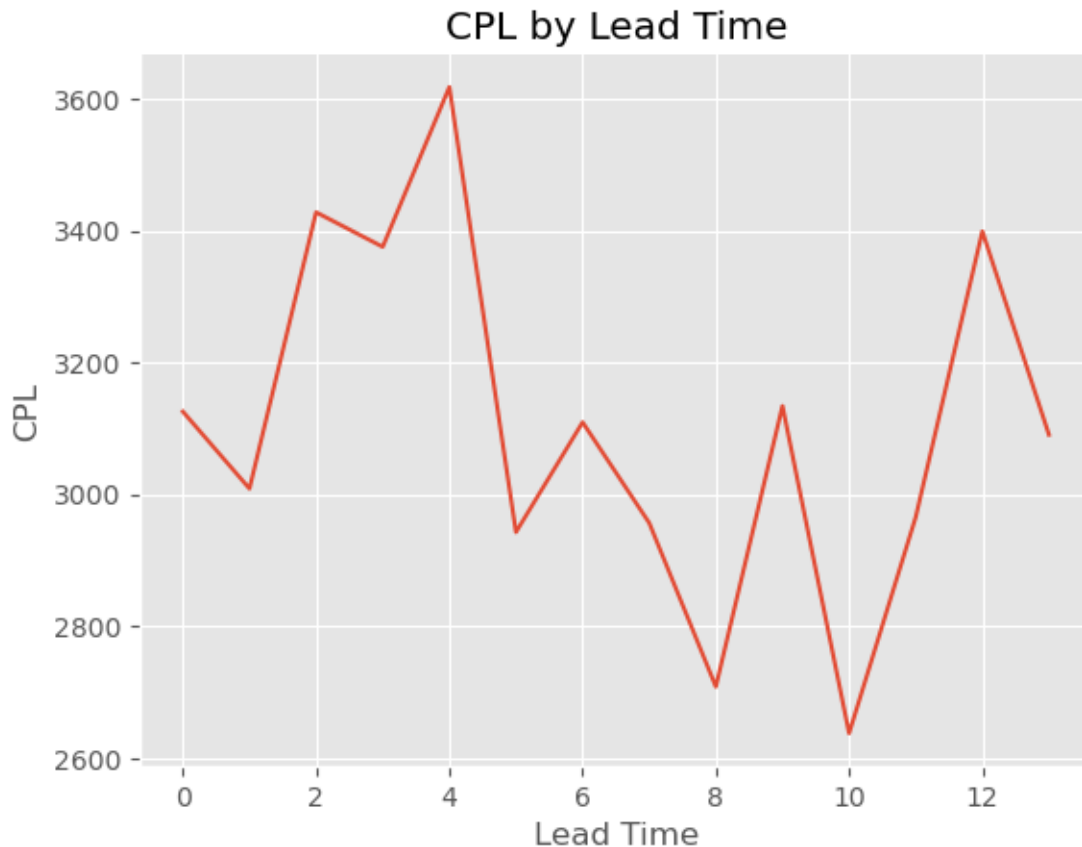
2.4 Features and target

```
[9]: x = df.drop(columns=["cpl", "lead_time"])
      y = df.cpl
```

3 Assumptions

1. All independent variables are necessary to be used in the models from the business logic perspective regardless of their parameter statistics. Explainability and prediction power are more significant in this regard.
2. The feature lead time is considered as ratio though the feature is discrete with only 13 unique entries. Because there is no meaningful and consistent change of slope in the graph below, "cpl by lead_time", that would indicate some sort of breakpoints to create categories upon.

```
[10]: lead_time = df.groupby("lead_time")["cpl"].mean()
      lead_time.plot.line(ylabel="CPL", xlabel="Lead Time", title="CPL by Lead Time");
```



4 ML - 1

Initial conditions

- Polynomial degree: 2, NO scaling, Linear Regression model --> one that could be done in MS e
- Lead time is rendered negative above to force all coefficients to be positive below
- Scaling is not considered in order to ease explainability and refrain from unscaling in ML-1

4.1 Train - test split

```
[11]: strats = df["trail_length"].astype(str) + "_" + df["equipment"].astype(str)
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.1,
random_state=42, stratify=strats)
```

4.2 Pipe

```
[12]: input_steps = [
    ("pol", PolynomialFeatures(degree=1, include_bias=False)),
    ("model", LinearRegression(positive=True))
]
```

```
pipe = Pipeline(input_steps)
```

4.3 Cross Validation

```
[13]: results = cross_validate(
    pipe,
    x_train,
    y_train,
    cv=5,
    scoring="r2",
    return_estimator=True,
    return_train_score=True
)

train_scores = results["train_score"]
validation_scores = results["test_score"]

scores = pd.DataFrame({
    "train_scores": train_scores,
    "validation_scores": validation_scores
})
display(scores)

print("Train scores average:", train_scores.mean())
print("Validation scores average:", validation_scores.mean())

print("Train scores standard deviation:", train_scores.std())
print("Validation scores standard deviation:", validation_scores.std())
```

	train_scores	validation_scores
0	0.817764	0.891624
1	0.819044	0.871906
2	0.833506	0.829748
3	0.850651	0.713806
4	0.858736	0.686125

Train scores average: 0.8359404663009282

Validation scores average: 0.7986417906035402

Train scores standard deviation: 0.016479484849093562

Validation scores standard deviation: 0.08347216606516682

4.4 Interpretation before further steps

1. Variance of train scores is low. Data character should be the same in every part of data.
2. Variance of validation scores is moderate. Overfitting test:

- overfitting ratio = $\frac{\text{Score}_{\text{train}} - \bar{\text{Score}}_{\text{val}}}{\text{Score}_{\text{train}}}$

```
[14]: overfitting_ratio = [round((train_score - validation_scores.mean()) /
    ↪train_score, 4) for train_score in train_scores]
display(pd.Series(overfitting_ratio).to_frame(name="Overfitting Ratios"))
print("Overfitting ratio average is: ", sum(overfitting_ratio) /
    ↪len(overfitting_ratio))
```

```
Overfitting Ratios
0          0.0234
1          0.0249
2          0.0418
3          0.0611
4          0.0700
```

Overfitting ratio average is: 0.04424

- overfitting ratio is less than 0.1 for all train scores and 0.05 in average. GOOD!.
- consistency ratio (CV) = $\frac{\sigma_{testscores}}{\mu_{testscores}}$

```
[15]: CV = validation_scores.std() / validation_scores.mean()
print("CV is: ", round(CV, 4))
```

CV is: 0.1045

- CV is 0.1. Just the threshold. GOOD!.

3. As CV sits just on the threshold, 0.1, the same model was run with polynomial degree 2. Comparison:

Measure (R^2)	degree = 2	degree = 1	Remarks
Train Scores	0.88	0.84	better in degree=2
Average			
Train Scores	0.01	0.02	better in degree=2
standard Deviation			but still low
Validation Scores	0.76	0.80	better in degree=1
Average			
Validation Scores	0.05	0.08	better in degree=2
Standard Deviation			
Overfitting Ratio	0.13	0.04	better in degree=1
			and less than 0.1
CV	0.07	0.1	better in degree=2
			but still within the limits

4. The run results of degree=1 and binary lead time alternative (taken from a makeshift notebook) are below. The improvement is so slight, negligible.

Measure (R^2)	degree = 1	degree = 1 and binary lead time (0, 4) - (>4)	Remarks
Train Scores Average	0.8358	0.8373	slightly better
Train Scores standard Deviation	0.0165	0.163	slightly better
Validation Scores Average	0.7997	0.8031	slightly better
Validation Scores Standard Deviation	0.084	0.0803	slightly better
Overfitting Ratio	0.0428	0.0404	slightly better
CV	0.1050	0.0999	slightly better

4.5 Decision

- Go with the degree=1 as this configuration provides less variance and better R2 score average in test scores with better generalization.

4.6 Fit, coefficients, predict, residuals

```
[16]: pipe.fit(x_train, y_train)
      coefs = pd.DataFrame({
          "features": pipe[:-1].get_feature_names_out(),
          "coefficients": pipe[-1].coef_
      })
      coefs
```

```
[16]:      features  coefficients
0    distance    1.553278
1  trail_length    0.000000
2      weight    0.000000
3   equipment    21.391564
4   lead_time1    22.145890
```

- As seen, trail_length and weight have zero contribution to the cpl.

```
[17]: y_pred = pipe.predict(x_test)
      y_pred_train = pipe.predict(x_train)
      residuals = y_test - y_pred
      print("Residuals average: ", residuals.mean())
```

Residuals average: 57.557197597135385

```
[18]: relative_bias = residuals.mean() / y_test.mean()
      print("Relative bias: ", relative_bias)
```

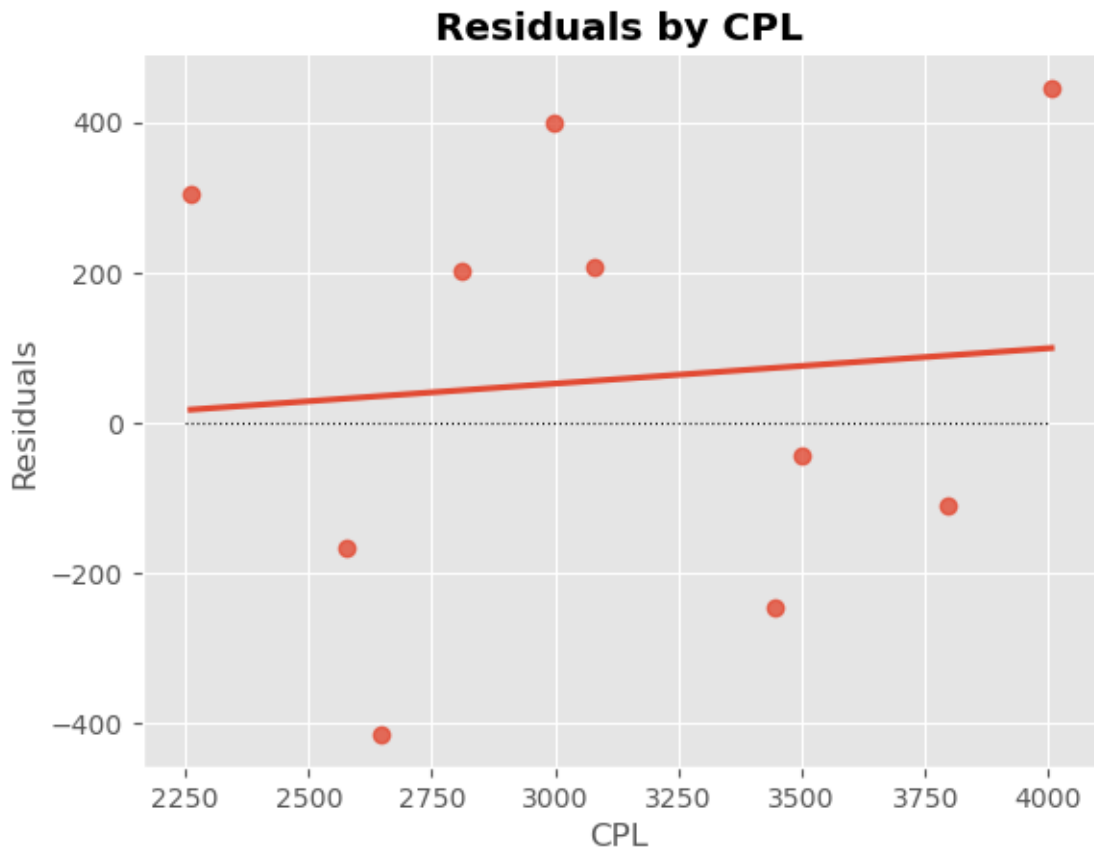
Relative bias: 0.018494054879871276

```
[19]: test_r2 = r2_score(y_test, y_pred)
      train_r2 = r2_score(y_train, y_pred_train)

      print("Test R2 score: ", test_r2)
      print("Train R2 score: ", train_r2)
```

Test R2 score: 0.7171949152213056
 Train R2 score: 0.8347591369367391

```
[20]: sns.regplot(y=residuals, x=y_test, order=1, ci=None)
      plt.ylabel("Residuals")
      plt.xlabel("CPL")
      plt.title("Residuals by CPL", fontweight="bold")
      plt.hlines(y=0, xmin=2250, xmax=4000, color="black", lw=0.8, ls=":");
```



4.6.1 Residual validation

- Quantitative validation of residuals will not be performed as there are only 10 values.
- Though this is also not quite reasonable to perform, qualitative validation:
 - Normality: cannot be done either due to scarce data

- Linearity: randomly dispersed 5 positive and 5 negative data points. Trendline above is misleading as standard error and variance is to be high with such scarce data: this is the reason behind the trendline's positive slope.
- Independence: some negative auto correlation seems to exist
- Equal variance: no variance change in residual values as the cpl values increased / decreased.

4.7 Insights

- Residuals mean is NOT zero. However, given the observation count is only 100 and test size is 10; residuals could not have enough test size to settle down and reach consistency.
- Moreover, relative bias (the ratio of average residuals to average target test values) is 1.8% only, less than 5%. That's acceptable.
- Test R^2 score is 72%, can be acceptable in supply chain management.

5 ML - 2 (Hyperparametrization)

- polynomial degrees
- polynomial interaction between independent variables
- scaling or NOT
- regularization (alphas)

5.1 Train Test Split

- Will be used as it is in ML-1

5.2 Pipe

```
[21]: input_steps = [
        ("pol", PolynomialFeatures(include_bias=False)),
        ("scaler", StandardScaler()),
        ("model", LassoCV(alphas = np.logspace(-3, 3, 5), positive=True,
        ↪max_iter=100000))
    ]

    pipe = Pipeline(input_steps)
```

- Intermediate Step : Hyperparameter Preparation

```
[22]: param_grid = [
        {
            "pol__degree": [1],
            "scaler": [StandardScaler(), "passthrough"]
        },
        {
            "pol__degree": [2, 3],
            "pol__interaction_only": [True, False],
            "scaler": [StandardScaler(), "passthrough"]
        }
    ]
```

```

    }
]

grid_search = GridSearchCV(
    estimator=pipe,
    param_grid=param_grid,
    cv=5,
    scoring="r2",
    n_jobs=-1
)

```

5.3 Cross Validation

```

[23]: results = cross_validate(
    grid_search,
    x_train,
    y_train,
    cv=5,
    scoring="r2",
    return_estimator=True,
    return_train_score=True,
    n_jobs=-1
)

```

```

[24]: train_scores = results["train_score"]
test_scores = results["test_score"]

scores = pd.DataFrame({
    "train_scores": train_scores,
    "validation_scores": test_scores}
)
display(scores)

print("Train scores average:", train_scores.mean())
print("Validation scores average:", test_scores.mean())

print("Train scores standard deviation:", train_scores.std())
print("Validation scores standard deviation:", test_scores.std())

```

	train_scores	validation_scores
0	0.829945	0.880792
1	0.818798	0.875062
2	0.831176	0.839693
3	0.846754	0.694611
4	0.852897	0.713999

```

Train scores average: 0.835914037180191
Validation scores average: 0.800831256879268

```

Train scores standard deviation: 0.012304080549344992

Validation scores standard deviation: 0.0802955415997817

- As seen above, no benefit is acquired from the hyperparametrization, the scores are the same as / even worse than the ones obtained in the ML-1 linear regression model
- Nevertheless, we will continue with fitting the model here and prove that the best parameters yielding the results above are either the ones used in ML-1 linear regression model or contributing with near-zero coefficients if ever different.

5.4 Fit, coefficients, predict, residuals

```
[25]: grid_search.fit(x_train, y_train)
```

```
[25]: GridSearchCV(cv=5,
                  estimator=Pipeline(steps=[('pol',
                                             PolynomialFeatures(include_bias=False)),
                                             ('scaler', StandardScaler()),
                                             ('model',
                                              LassoCV(alphas=array([1.00000000e-03,
3.16227766e-02, 1.00000000e+00, 3.16227766e+01,
1.00000000e+03])),
                                             max_iter=100000,
                                             positive=True))]),
                  n_jobs=-1,
                  param_grid=[{'pol__degree': [1],
                              'scaler': [StandardScaler(), 'passthrough']},
                              {'pol__degree': [2, 3],
                               'pol__interaction_only': [True, False],
                               'scaler': [StandardScaler(), 'passthrough']}],
                  scoring='r2')
```

```
[26]: best_pipe = grid_search.best_estimator_

coefs = pd.DataFrame({
    "features": best_pipe[:-1].get_feature_names_out(),
    "coefficients": best_pipe[-1].coef_
})

degree = best_pipe["pol"].degree
print("Best pollynomial degree: ", degree)

interaction = best_pipe["pol"].interaction_only
print("Interaction used?: ", interaction)

scaling = best_pipe["scaler"]
print("Scaling used?: ", scaling if not scaling=="passthrough" else False)

alpha = best_pipe["model"].alpha_
```

```
print("Regularization used: ", round(alpha, 2))

coefs
```

```
Best pollynomial degree: 2
Interaction used?: False
Scaling used?: False
Regularization used: 31.62
```

```
[26]:
```

	features	coefficients
0	distance	1.608893
1	trail_length	0.000000
2	weight	0.011220
3	equipment	0.000000
4	lead_time1	0.000000
5	distance^2	0.000000
6	distance trail_length	0.056606
7	distance weight	0.000000
8	distance equipment	0.002431
9	distance lead_time1	0.012435
10	trail_length^2	0.000000
11	trail_length weight	0.000000
12	trail_length equipment	0.000000
13	trail_length lead_time1	11.946831
14	weight^2	0.000000
15	weight equipment	0.000000
16	weight lead_time1	0.002516
17	equipment^2	0.000000
18	equipment lead_time1	0.000000
19	lead_time1^2	5.271451

- As seen above, the polynomial degree is 2 (not 1) but it is only used for square lead_time.
- Trail length, weight and lead time is not used in degree 1.
- There are some interactions used but with near-zero values.
- Scaling is not used as it is in the ML-1.

```
[27]: y_pred = grid_search.predict(x_test)
```

```
[28]: residuals = y_test - y_pred
```

```
[29]: print("Residuals average: ", residuals.mean())
relative_bias = residuals.mean() / y_test.mean()
print("Relative bias: ", relative_bias)
```

```
Residuals average: 94.65276210994594
Relative bias: 0.030413457396679503
```

- Residuals average and the relative bias here is 1.65 times that of ML-1. BAD!

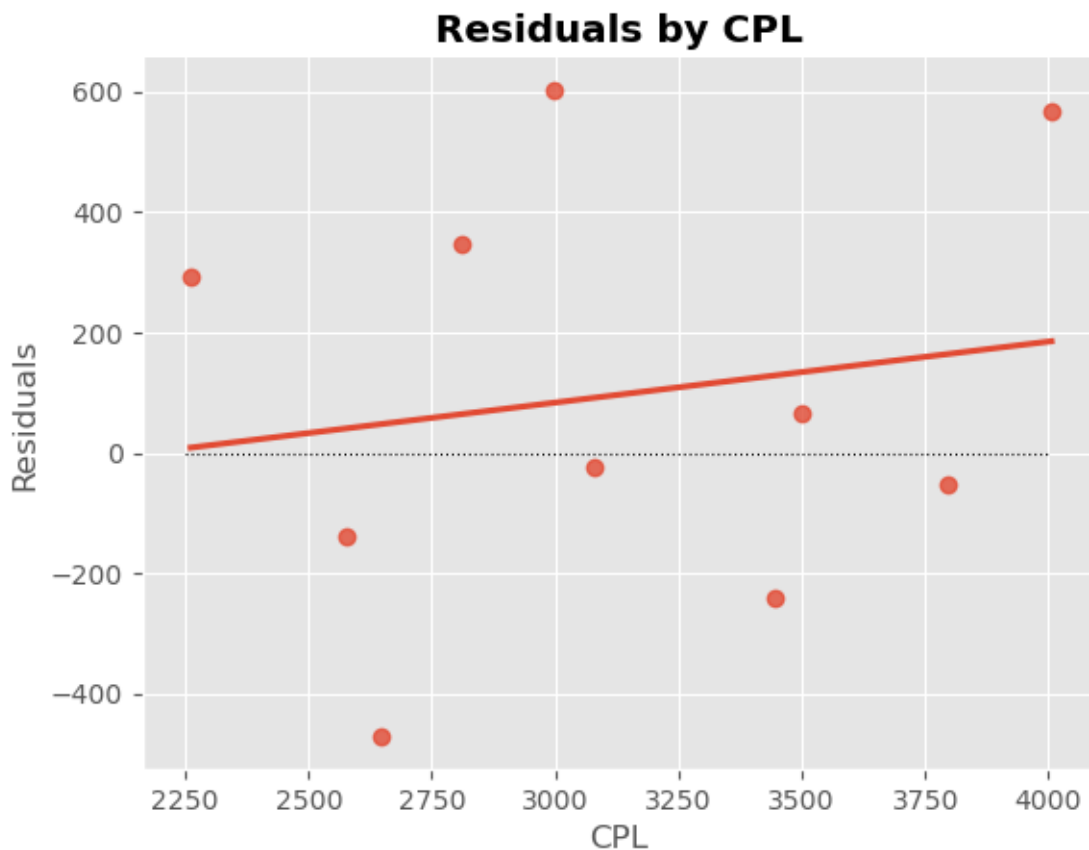
```
[30]: test_r2 = r2_score(y_test, y_pred)
      train_r2 = r2_score(y_train, y_pred_train)

      print("Test R2 score: ", test_r2)
      print("Train R2 score: ", train_r2)
```

Test R2 score: 0.5814072281895868
 Train R2 score: 0.8347591369367391

- Test R2 score is 20% less than that of ML-1. BAD!

```
[31]: sns.regplot(y=residuals, x=y_test, order=1, ci=None)
      plt.ylabel("Residuals")
      plt.xlabel("CPL")
      plt.title("Residuals by CPL", fontweight="bold")
      plt.hlines(y=0, xmin=2250, xmax=4000, color="black", lw=0.8, ls=":");
```



5.4.1 Residual validation

- Quantitative validation of residuals will not be performed as there are only 10 values.
- Though this is also not quite reasonable to perform, qualitative validation:

- Normality: cannot be done either due to scarce data
- Linearity: randomly dispersed 5 positive and 5 negative data points. Trendline above is misleading as standard error and variance is to be high with such scarce data: this is the reason behind the trendline's positive slope.
- Independence: some negative auto correlation seems to exist
- Equal variance: no variance change in residual values as the cpl values increased / decreased.

6 Final Word

Simpler the Better